

บทที่ 3

โครงสร้างและการทำงานของโปรแกรม

ในโปรแกรม นี้จะมีอยู่สองส่วนหลักๆ คือ ส่วนของการค้นหาองค์ประกอบของใบหน้า และ ส่วนของการจัดกลุ่มพีเจอรันนิวอลเน็ตเวิร์ค

ในบทที่นี้ 3 จะกล่าวถึง โครงสร้างของระบบค้นหาองค์ประกอบของใบหน้า การสอนนิวอลเน็ตเวิร์ค การแบ่งกลุ่มโดยองค์ประกอบของใบหน้า การสร้างคลัสเตอร์นิวอลเน็ตเวิร์ค

3.1 การสอนนิวอลเน็ตเวิร์คแบบแพร่กลับ

เมื่อได้ชุดข้อมูลขององค์ประกอบของใบหน้าแล้ว จะนำเข้ามาสอนนิวอลเน็ตเวิร์ค ซึ่งนิวอลเน็ตเวิร์คที่ใช้จะเป็นนิวอลเน็ตเวิร์คแบบแพร่กลับ (Back-propagation Neural Network) ซึ่งถือเป็นรูปแบบทั่วไปที่นิยมกัน และนิวอลเน็ตเวิร์คแบบแพร่กลับนี้พัฒนาขึ้นมาจากโปรแกรม ภาษา C# เพื่อที่จะได้สามารถนำมาสร้างเป็นคลัสเตอร์นิวอลเน็ตเวิร์คต่อไป

ในการสร้างนิวอลเน็ตเวิร์คนี้จะสร้าง อินพุตยูนิท ขึ้นมาโดยให้มีจำนวนเท่ากับองค์ประกอบของภาพที่ค้นหาได้ จากนั้นจึงสร้าง เอาพุตยูนิท ขึ้นมาเท่ากับจำนวนค่าที่ต้องการ โดยในโปรแกรมนี้อาจแบ่งออกมาเป็น ชาย หรือ หญิง จึงมีเอาพุตยูนิทเพียง 1 ยูนิท โดยค่าที่ได้จากนิวอลเน็ตเวิร์คจะกำหนดให้ ชายเป็น 1 และหญิงเป็น 0

จากนั้นจะทำการสุ่มค่าของน้ำหนักของการเชื่อมต่อและค่าเปรียบเทียบกับ (Threshold)

แล้วจึงทำการสอนนิวอลเน็ตเวิร์ค โดยการนำองค์ประกอบของใบหน้าเข้ามาในนิวอลเน็ตเวิร์คทีละชุด มาเป็นค่าเอาพุตของอินพุตยูนิท แล้วอิดเคนยูนิทหาค่าเอาพุตจากสมาการ

$$f(\alpha) = \frac{1}{1 + e^{-(\alpha - \theta)}} \quad (3.1)$$

$$\alpha = \sum_{j=1}^m (X_i \times W_{ij}) \quad (3.2)$$

เมื่อ X_i เป็นเอาพุตจากยูนิท i ในชั้นก่อนหน้า

W_{ij} เป็นค่าน้ำหนักจากยูนิท i มายังยูนิท j

เพื่อส่งออกไปเป็นเอาพุตของอิดเคนยูนิท ในแต่ละเอาพุตของแต่ละยูนิท และเอาพุตยูนิทก็จะหาค่าเอาพุตด้วยวิธีเดียวกัน โดยเอาพุตของเอาพุตยูนิทนี้จะมีค่าเป็นเอาพุตของนิวอลเน็ตเวิร์คด้วย

เมื่อได้ค่าเอาพุทของนิวรอลเน็ตเวิร์คแล้ว จะทำการปรับค่าน้ำหนักใหม่ตามสมการดังต่อไปนี้

$$w_{ij}(p+1) = w_{ij}(p) + \Delta w_{ij}(p) \quad (3.3)$$

$$\Delta w_{ij}(p) = \alpha + x_i(p) + \delta_j(p) \quad (3.4)$$

$$\delta_j(p) = y_j(p) \times [1 - y_j(p)] \times e \quad (3.5)$$

เมื่อ	$w_{ij}(p)$	เป็นค่าน้ำหนักจาก i ไป j ในรอบที่ p
	$\Delta w_{ij}(p)$	เป็นค่าสำหรับปรับน้ำหนักจาก i ไป j
	α	เป็นค่าการเรียนรู้ซึ่งต้องกำหนดโดยผู้ใช้
	$\delta_j(p)$	เป็นค่าเกรเดียนต์ที่ใช้ในการปรับน้ำหนัก
	$x_i(p)$	เป็นค่าอินพุทของยูนิต i จากชั้นก่อนหน้า
	y_j	เป็นค่าเอาพุทของยูนิต j

ส่วนค่า e จะมีการคำนวณที่แตกต่างกันในเอาพุทยูนิตและฮิดเดนยูนิต โดยในเอาพุทยูนิตจะหาค่า e ดังนี้

$$e = y_{dk}(p) - y_k(p) \quad (3.6)$$

เมื่อ	e	เป็นค่าคาดเคลื่อนของผลลัพธ์ของเน็ตเวิร์ค
	$y_{dk}(p)$	เป็นผลลัพธ์ที่ถูกต้อง
	$y_k(p)$	เป็นผลลัพธ์ที่ได้ออกมาจากเอาพุทยูนิต

และในฮิดเดนยูนิตจะหาค่า e ดังนี้

$$e = \sum_k \delta_k(p) \times w_{jk}(p) \quad (3.7)$$

เมื่อ	$\delta_k(p)$	เป็นค่าเกรเดียนต์ k ที่คำนวณได้ชั้นถัดไปที่เชื่อมต่อกันฮิดเดนยูนิต j
	$w_{jk}(p)$	เป็นน้ำหนักของชั้นถัดไปที่เชื่อมต่อกันฮิดเดนยูนิต j

แล้ววนซ้ำไปเรื่อยๆ จนกว่าจะมีค่า SSE (sum square error) ต่ำกว่า 0.001 โดยค่า SSE นี้หาได้จากสมการ

$$SSE = \sum (y_{dk}(p) - y_k(p))^2 \quad (3.8)$$

เมื่อ $y_{dk}(p)$ เป็นผลลัพธ์ที่ถูกต้อง

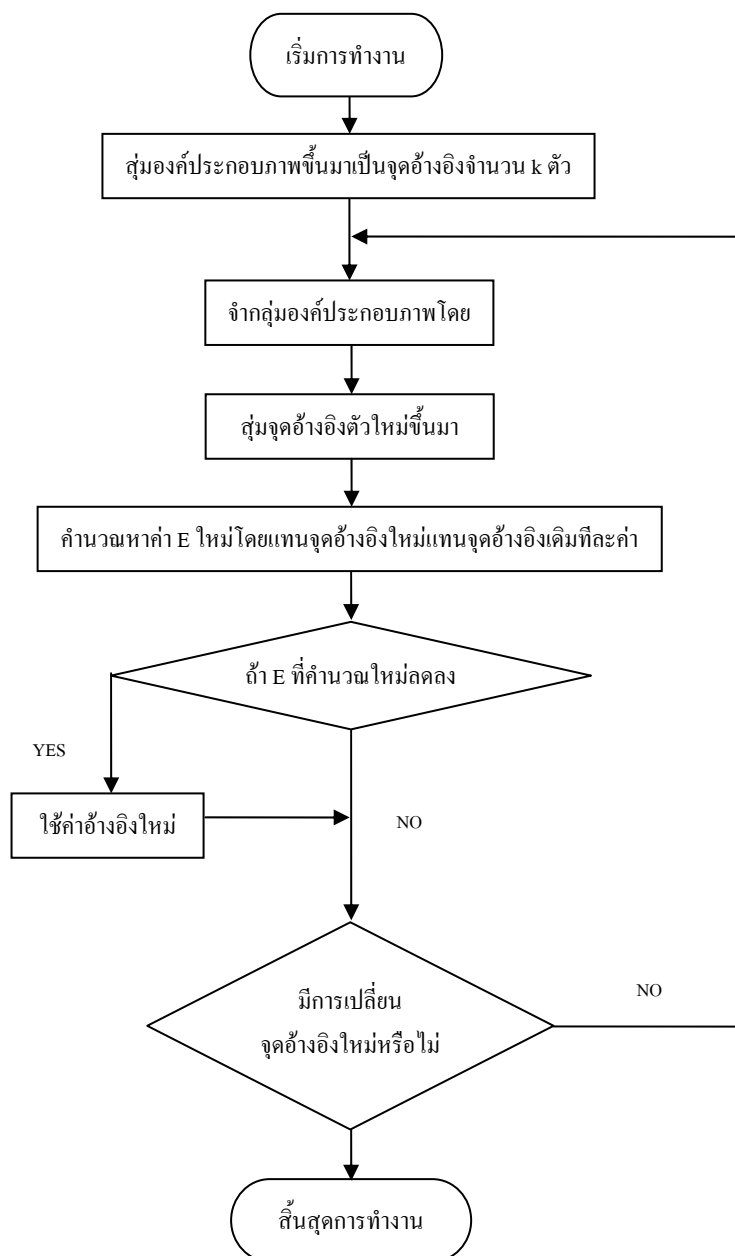
$y_k(p)$ เป็นผลลัพธ์ที่ได้ออกมาจากเอาพุตยูนิต

และเมื่อได้ค่า SSE ต่ำกว่า 0.001 แล้วก็จะถือว่าเสร็จสิ้นการสอนนิรอลเน็ตเวิร์ค

ส่วนการทำงานของนิรอลเน็ตเวิร์คก็จะเป็นการหาค่าเอาพุตออกมา โดยไม่มีการกลับไปปรับค่าน้ำหนัก

3.2 การจับกลุ่มฟีเจอร

จะทำการจับกลุ่มโดยแทนที่ฟีเจอรแต่ละอย่าง ด้วยอินพุตยูนิต ของนิรอลเน็ตเวิร์ค ซึ่งในการจับกลุ่มนี้จะดูจากค่าน้ำหนัก (weight) ที่อยู่บนการเชื่อมต่อจากอินพุตยูนิตนั้นๆ ไปยังอิดเดนยูนิตในชั้นถัดไปของนิรอลเน็ตเวิร์คแบบแพร่กลับ (Back-propagation Neural Network) ที่ผ่านการสอนแล้ว และในการจับกลุ่มนี้ใช้อัลกอริทึมที่ K-Medoid มาจับกลุ่มซึ่งสามารถเขียนเป็น Flow Chart ได้ดังนี้



รูปที่ 3.1 Flow chart การจับกลุ่มองค์ประกอบใบหน้า

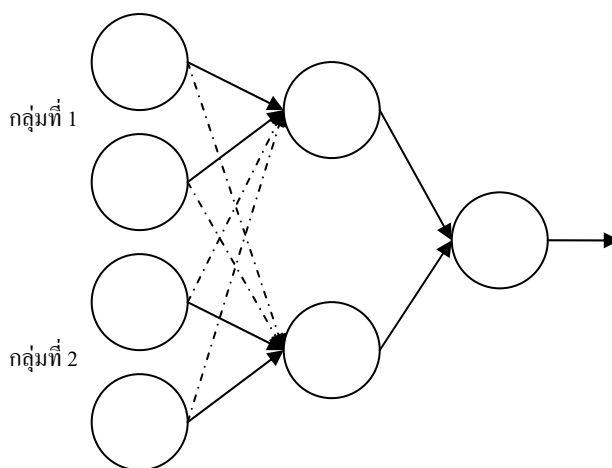
โดยในการจับกลุ่มนี้เกิดจากการสุ่มจึงอาจเกิดความผิดพลาดขึ้นได้ จึงต้องมีการทำซ้ำประมาณ 100-1000 รอบเพื่อที่จะลดความผิดพลาดในการจับกลุ่มลง

Cluster								
hidden 0	hidden 1	hidden 2	hidden 3	hidden 4	hidden 5	hidden 6	hidden 7	group
-5.495053883	11.55363014	0.739140321	0.472798990	0.389455582	-0.098050516	2.132998843	4.215533649	0
-5.785406929	0.807138414	1.276735631	1.622125621	1.812895579	-0.871095710	0.759801460	0.663238155	0
11.12773060	-15.33675759	-2.600407504	-2.732865866	-2.773481110	1.284449107	-3.713001913	-6.276653802	1
11.64650389	-15.14455087	-1.520821560	-1.548066931	-1.410606833	0.530864706	-3.665539901	-6.579911493	1

รูปที่ 3.2 ผลลัพธ์การจับกลุ่มนำหน้าของนิรอลเน็ตเวิร์คที่สอนด้วยข้อมูลของ Iris

3.3 การสร้างและสอนคลัสเตอร์นิวรอลเน็ตเวิร์ค (Cluster Neural Network)

ในการสร้างคลัสเตอร์นิวรอลเน็ตเวิร์คนี้ จะสร้างขึ้นโดยใช้ผลจากการจับกลุ่มอินพุตยูนิท (ใช้ผลจากการจับกลุ่มขององค์ประกอบใบหน้า) มาใช้ในการตัดสินใจเชื่อมต่อที่ไม่เกี่ยวข้องกัน โดยให้อินพุตยูนิทในกลุ่มเดียวกันเชื่อมต่อไปยังฮิดเดน ยูนิทในกลุ่มเดียวกันเท่านั้น ดังภาพต่อไปนี้



รูปที่ 3.3 แบบจำลองของคลัสเตอร์นิวรอลเน็ตเวิร์ค

รูปนี้จะแสดงนิวรอลเน็ตเวิร์คแบบคลัสเตอร์ โดยเส้นประจะแสดงถึงการเชื่อมต่อที่โดนตัดออกไป

จากนั้นจะทำการสุ่มค่าน้ำหนักของส่วนเชื่อมต่อที่เหลืออยู่ทั้งหมดขึ้นมาใหม่ แล้วจึงสอนคลัสเตอร์นิวรอลเน็ตเวิร์คด้วยวิธีการเดียวกับนิวรอลเน็ตเวิร์คแบบแพร่กลับ แต่ไม่

ในการใช้งานนิวรอลเน็ตเวิร์คนั้นจะคล้ายกับการสอนนิวรอลเน็ตเวิร์ค โดยจะต่างกันเพียงไม่จำเป็นต้องมีการปรับค่าน้ำหนักเท่านั้น

3.4 โครงสร้างของระบบค้นหาฟีเจอ์

ในระบบจัดกลุ่มฟีเจอ์นี้ มีการใช้ฟีเจอ์ของใบหน้าของมนุษย์ประกอบด้วย ดังนั้นการประมวลผลภาพจึงเข้ามามีบทบาท ในส่วนของการประมวลผลภาพนั้นมีขั้นตอนในการแยก ตา ปาก จมูก ออกจากข้อมูลภาพทั้งหมด โดยมีขั้นตอนการทำงานของโปรแกรมหาดังนี้คือ ในขั้นแรกจะเป็นขั้นตอนการเตรียมภาพโดยนำภาพถ่าย RGB มาแปลงเป็นภาพแบบไบนารีซึ่งจะเหลือข้อมูลภาพเพียงสองระดับ คือ 0 กับ 1 จากนั้นก็จะทำการกำจัดสัญญาณรบกวนแล้วทำการหาขอบของรูปภาพ แล้วจึงค่อยสแกนหา ตา ปาก และจมูก โดยจะพิจารณาจากตำแหน่งที่ควรจะเป็น แล้วจึงนำขนาดของฟีเจอ์เหล่านี้เข้าสู่การจำแนกต่อไป

3.4.1 การประมวลผลภาพ

ในการประมวลผลภาพภาพสามารถแบ่งการทำงานออกได้เป็น

1. การเตรียมข้อมูลภาพ
2. การแปลงข้อมูลภาพไปนารี
3. การหาตำแหน่งของดวงตา
4. การหาตำแหน่งของจมูกและปาก
5. การแยกตา ปาก จมูกออกจากภาพ

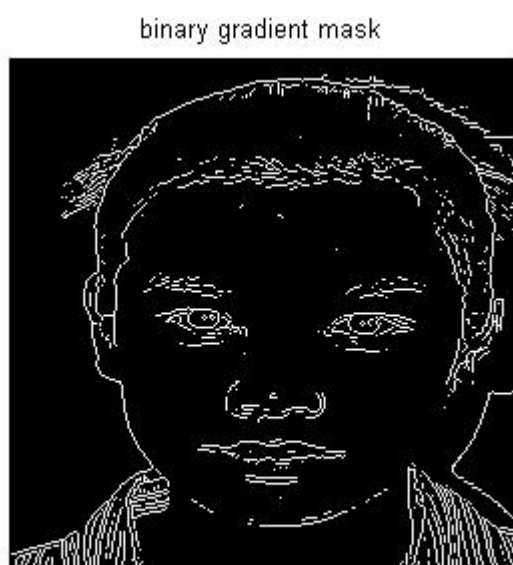
การแปลงข้อมูลภาพไปนารี

ในขั้นตอนแรกจะเป็นการเตรียมข้อมูลภาพเพื่อใช้ในการประมวลผล เนื่องรูปถ่ายที่ได้มาจะเป็นภาพสี RGB



รูปที่ 3.4 ภาพตัวอย่างรูปถ่ายจากกล้องดิจิทัล

เพื่อให้ข้อมูลภาพมีรูปแบบที่เหมาะสมในการประมวลผล จึงต้องทำการแปลงข้อมูลของภาพให้เป็นภาพไบนารี คือ มีระดับสีเพียง 2 ระดับได้แก่สีดำเป็น 1 และสีขาวเป็น 0 วิธีการแปลงข้อมูลภาพให้เป็นไบนารีนั้น ทำได้โดยการเลือกค่า threshold ที่เหมาะสม โดยข้อมูลภาพที่มีระดับความสว่างสูงกว่าค่า threshold จะถูกปรับเป็น 0 หรือสีขาว ส่วนค่าระดับความสว่างที่มีค่าต่ำกว่าค่า threshold จะถูกปรับเป็น 1 หรือสีดำ



รูปที่ 3.5 ภาพที่ได้จากการแปลงเป็นไบนารี

ภาพที่เป็นภาพไบนารีแล้วจะได้ ตา ปาก จมูก ออกจากใบหน้าเป็นส่วนๆ อย่างชัดเจนเนื่องมาจากการเลือกค่า threshold ที่เหมาะสม แต่อาจมีบางกรณี que เลือกค่า threshold สูงหรือต่ำเกินไปจะทำให้ไม่สามารถแยกแยะ ตา ปาก จมูก ออกจากใบหน้าได้หรือได้ขนาดที่ผิดพลาดไป ดังนั้นต้องเลือกค่า threshold ที่เหมาะสมที่สุดในการแปลงเป็นภาพไบนารีเพื่อให้เกิดความเสียหายของข้อมูลน้อยที่สุด

binary gradient mask



รูปที่ 3.6 แสดงการเลือกค่า threshold ที่ต่ำเกินไป

binary gradient mask



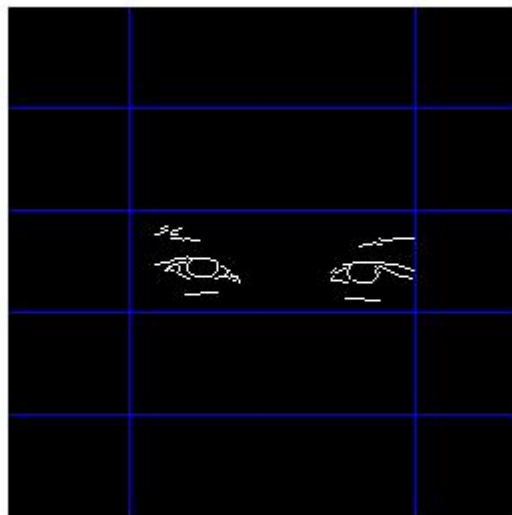
รูปที่ 3.7 แสดงการเลือกค่า threshold มากเกินไป

การกำจัดสิ่งรบกวน

ในขั้นตอนของการแปลงให้เป็นภาพไบนารีนี้อาจทำให้เกิดพิกเซลสีดำเดี่ยวๆจำนวนมาก ซึ่งเป็นสิ่งรบกวน (Noise) ทำให้เกิดผลต่อการประมวลผลในขั้นตอนต่อไปได้ ดังนั้นจึงต้องกำจัดพิกเซลที่เป็นสิ่งรบกวนเหล่านี้ออกจากภาพ โดยการหาตำแหน่งของพิกเซลสีดำที่ไม่มีพิกเซลข้างเคียงเลย แล้วเปลี่ยนให้เป็นพิกเซลสีขาวแทน

การหาตำแหน่งของดวงตา

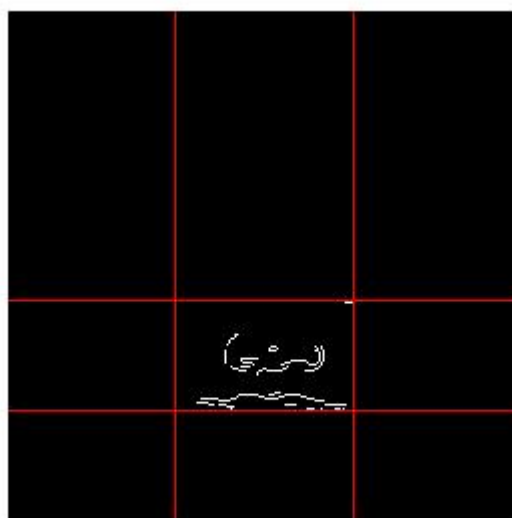
หลังจากที่ภาพได้ผ่านการแปลงให้เป็นไบนารีแล้ว สามารถหาตำแหน่งของดวงตาได้ โดยใช้วิธีการพิจารณาว่าดวงตาคือกลุ่มของพิกเซลสีดำหรือพิกเซลที่มีค่าเป็น 1 ติดกัน ดังนั้นภาพที่ผ่านการแปลงเป็นไบนารีแล้วนี้จะมีกลุ่มของพิกเซลดังกล่าวอยู่หลายกลุ่มทั้งที่เป็นดวงตา จมูก ปาก หรือส่วนอื่นๆของภาพ โดยที่ดวงตาควรจะอยู่ในกรอบสี่เหลี่ยม โดยในแนวแกน X=102-153 และแกน Y=61-204



รูปที่ 3.8 แสดงกรอบสี่เหลี่ยมที่ใช้ในการหาดวงตา

การหาตำแหน่งของจมูกและปาก

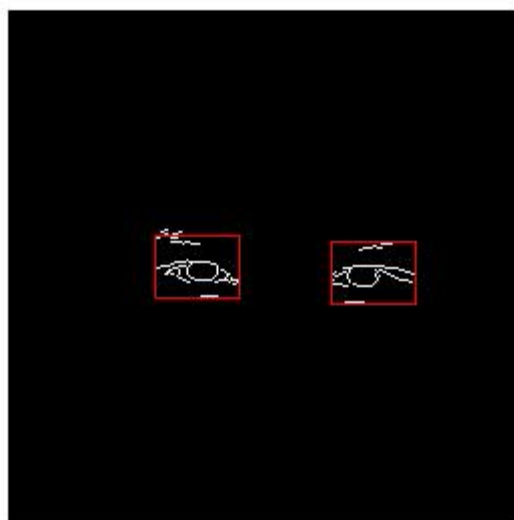
เช่นเดียวกับดวงตาจมูกและปากจะมีกลุ่มพิกเซลที่มีค่าเป็น 1 ติดกัน โดยตำแหน่งของจมูกและปากควรจะอยู่ในกรอบสี่เหลี่ยม โดยในแนวแกน X=145-200 และแกน Y=84-173



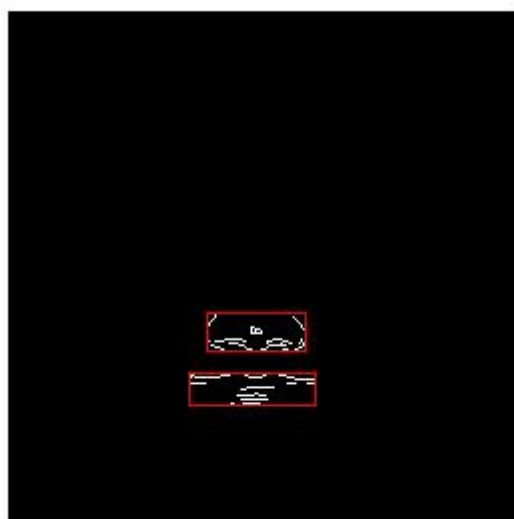
รูปที่ 3.9 แสดงกรอบสี่เหลี่ยมที่ใช้หาจมูกและปาก

การแยกตา ปาก จมูกออกจากภาพ

หลังจากที่รู้ตำแหน่งของฟีเจอร์ต่างๆแล้ว ก็จะทำการแยกแต่ละฟีเจอร์ออกจากภาพโดยการสแกนจากซ้ายไปขวาเพื่อหาขอบซ้าย สแกนจากบนลงล่างเพื่อหาขอบบน สแกนจากขวาไปซ้ายเพื่อหาขอบขวา และสแกนจากล่างขึ้นบนเพื่อหาขอบล่าง จากนั้นก็หาคำวนเวียนรอบขอบที่หาได้จะได้กรอบรอบฟีเจอร์เพื่อนำมาหาขนาดต่อไป



รูปที่ 3.10 แสดงขนาดของดวงตาที่หาได้

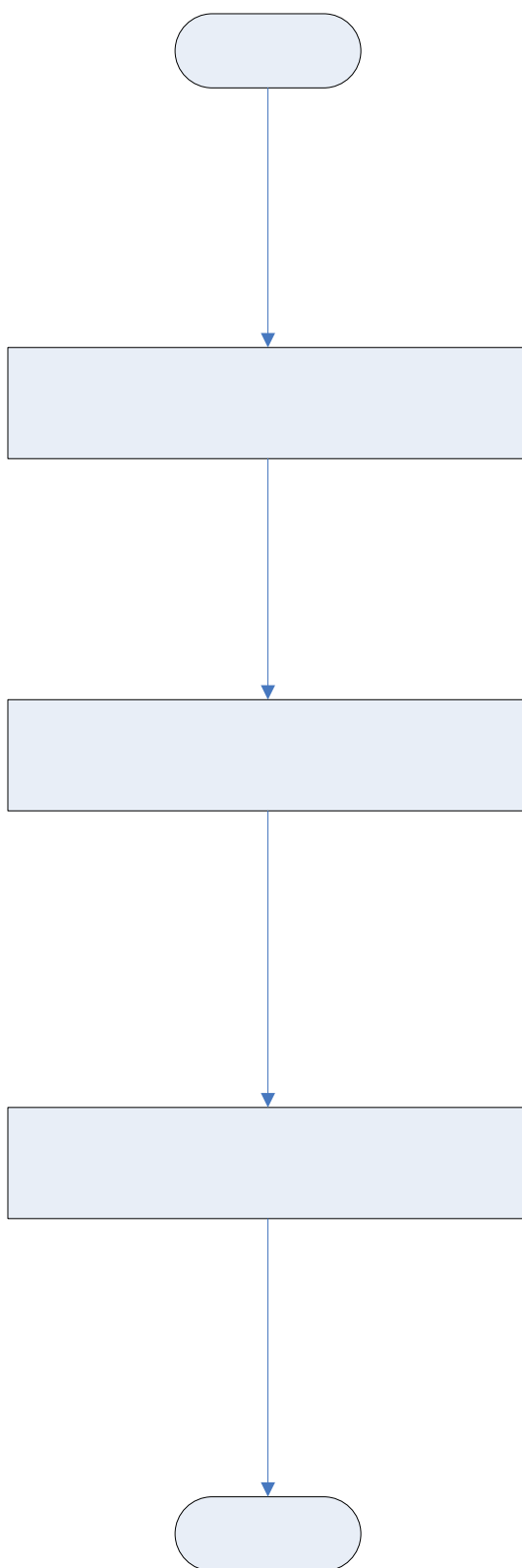


รูปที่ 3.11 แสดงขนาดของปากและจมูกที่หาได้

3.4.2 โครงสร้างของโปรแกรมส่วนค้นหาองค์ประกอบบนใบหน้า

โปรแกรม Face Classification แบ่งออกเป็น 2 ส่วน คือ ส่วนการประมวลผลภาพและส่วนของการจำแนกใบหน้า โดยในส่วนของการประมวลผลภาพนั้นเขียนขึ้นโดยภาษา Matlab ส่วนการจำแนกใบหน้าเขียนขึ้นโดยภาษา C# และทำงานร่วมกันโดยการ Compile M-File มาเป็นไฟล์สกุล dll เพื่อนำฟังก์ชันไปเรียกใช้ในการจำแนกใบหน้า โดยแบ่งโปรแกรมออกเป็นส่วนย่อยดังนี้

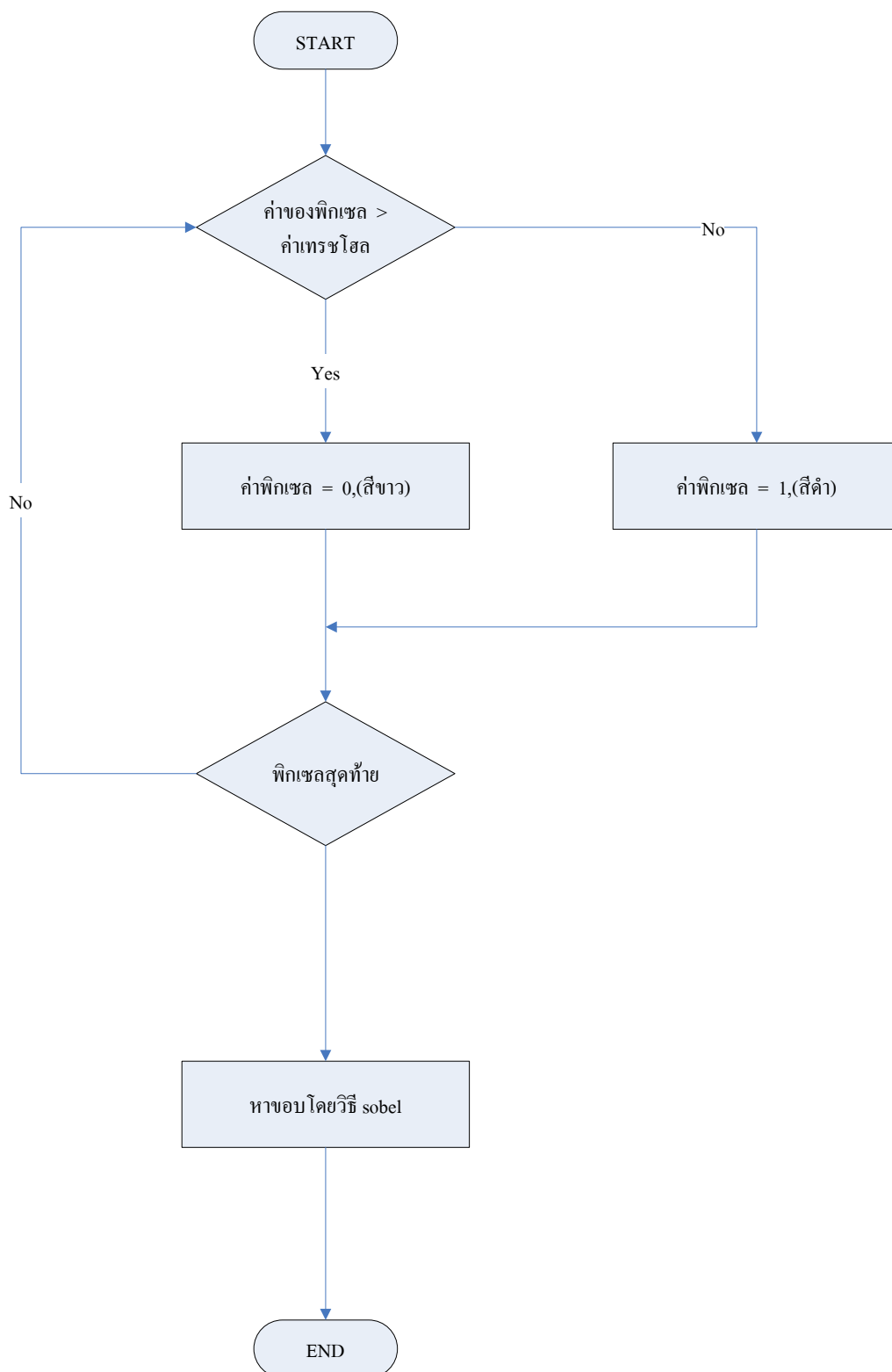
โปรแกรมการทำ Preprocessing



รูปที่ 3.12 Flow chart ของโปรแกรมทำ Preprocessing

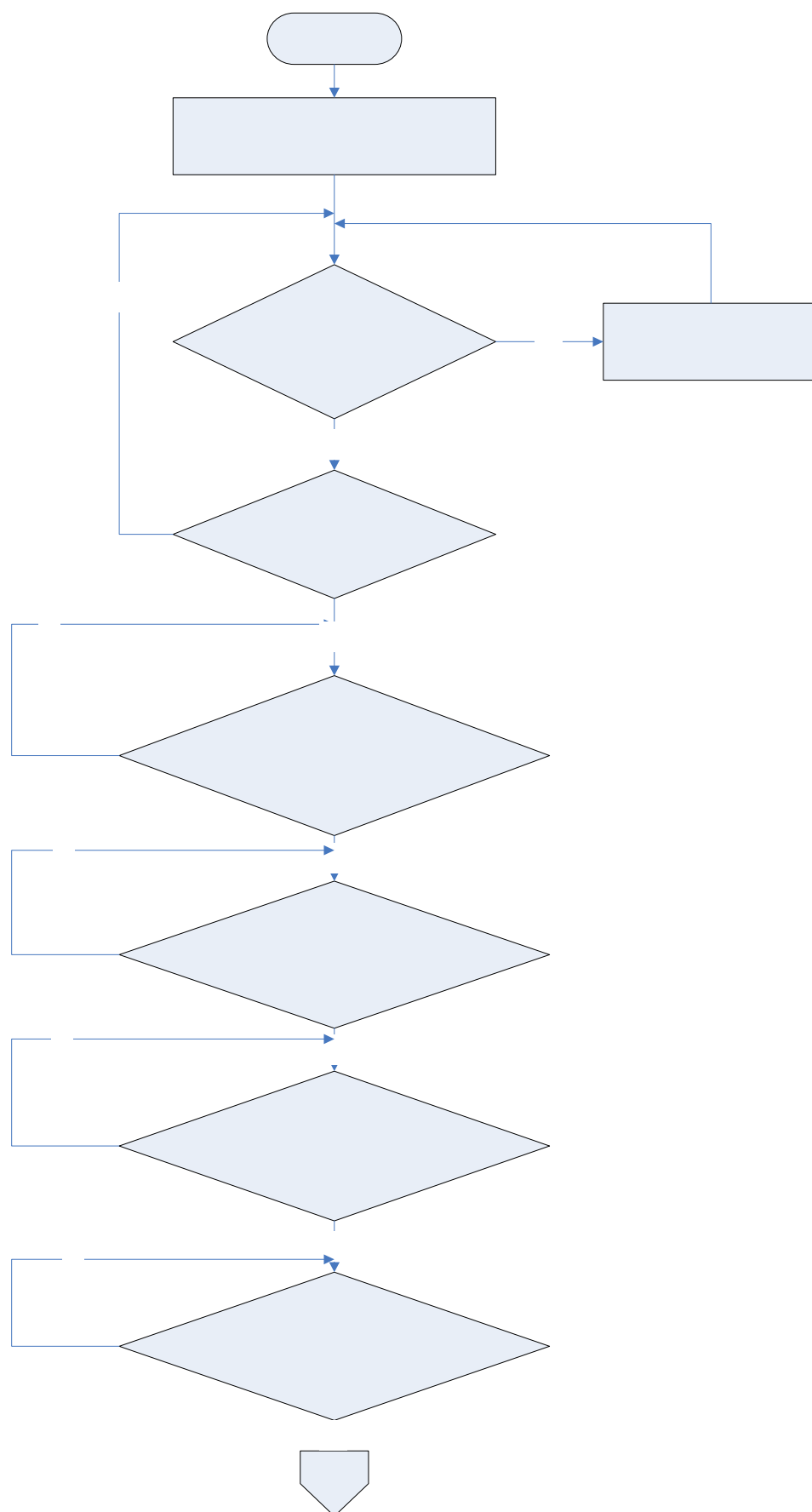
โปรแกรมการทำ Binarization

เมื่อทำการปรับปรุงภาพเบื้องต้นแล้วก็จะนำรูปที่ปรับปรุงมาแล้วนั้นมาแปลงเป็นภาพสีขาวดำ แล้วทำการหาขอบของภาพ



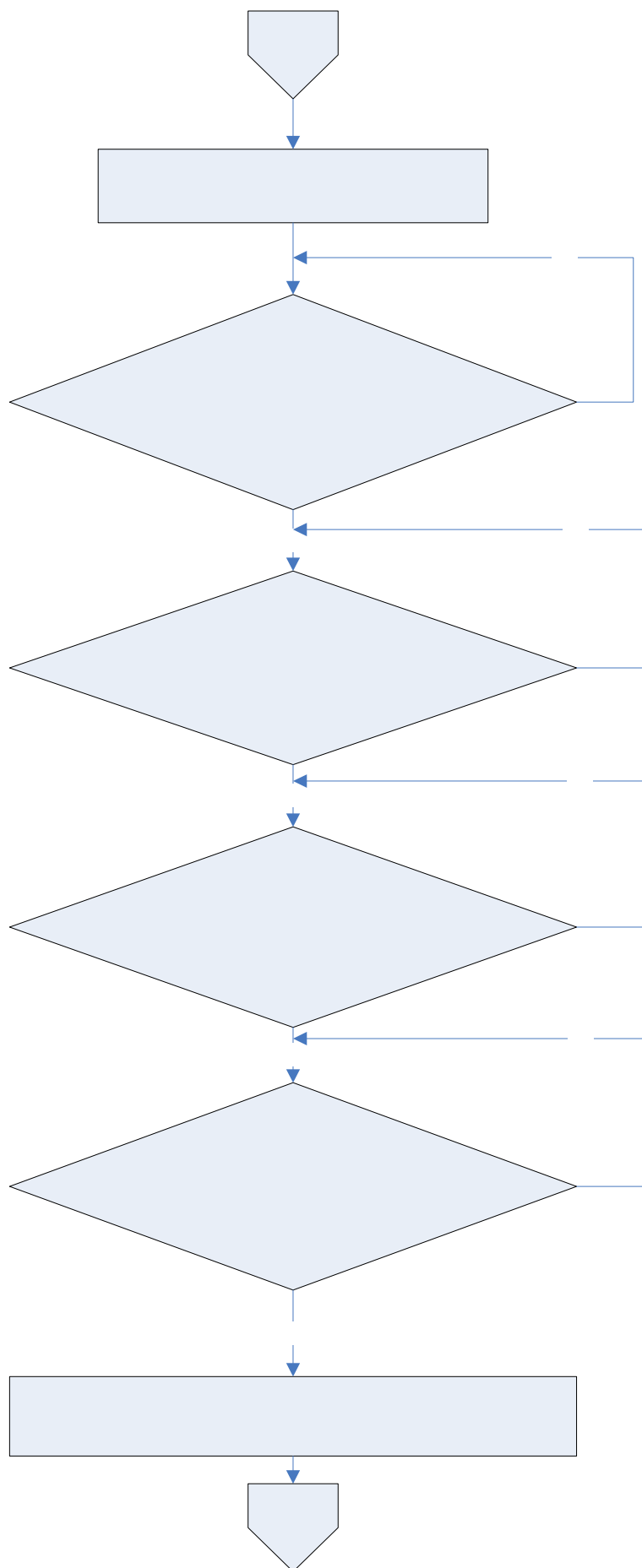
รูปที่ 3.13 Flow chart ของโปรแกรมการทำ Binarization

โปรแกรมการหาขนาดของตา

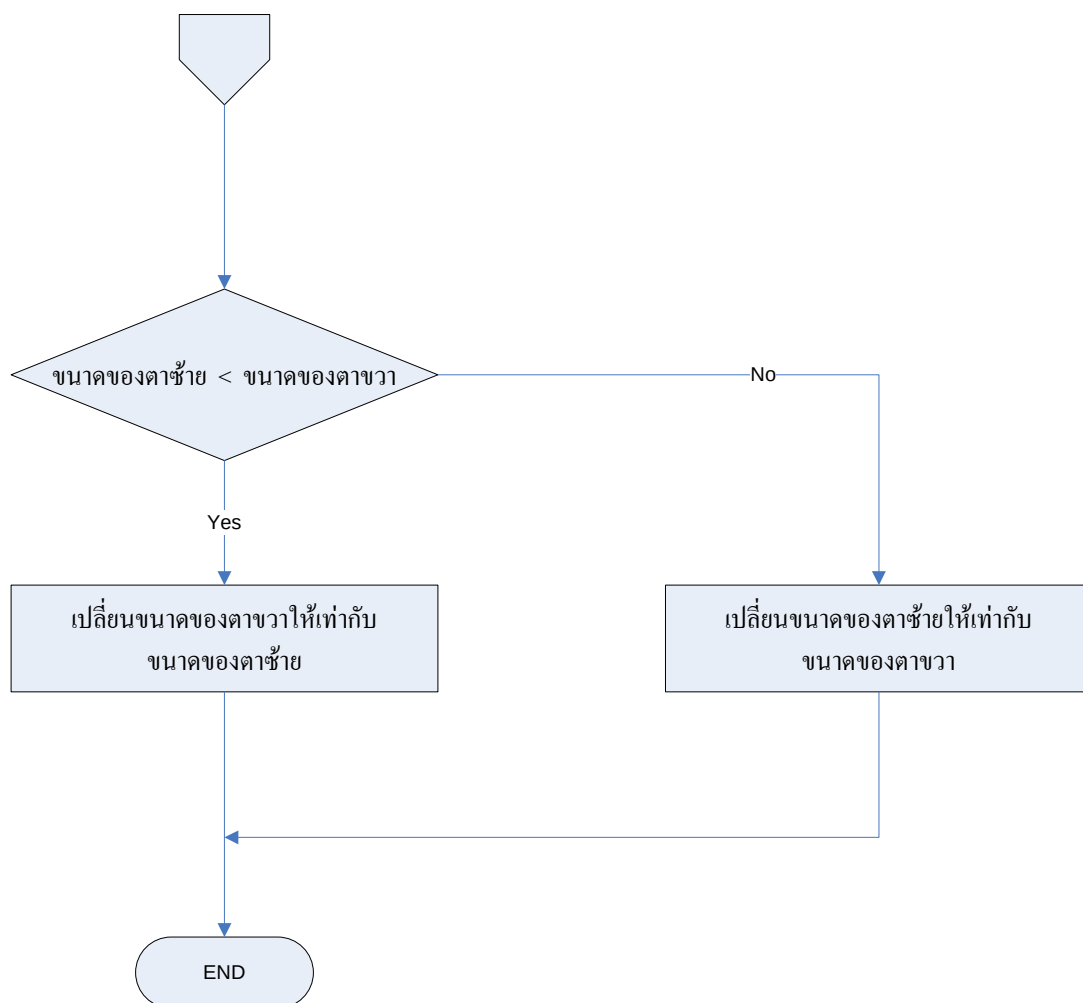


No

รูปที่ 3.14 Flow chart ของโปรแกรมการหาขนาดของตา

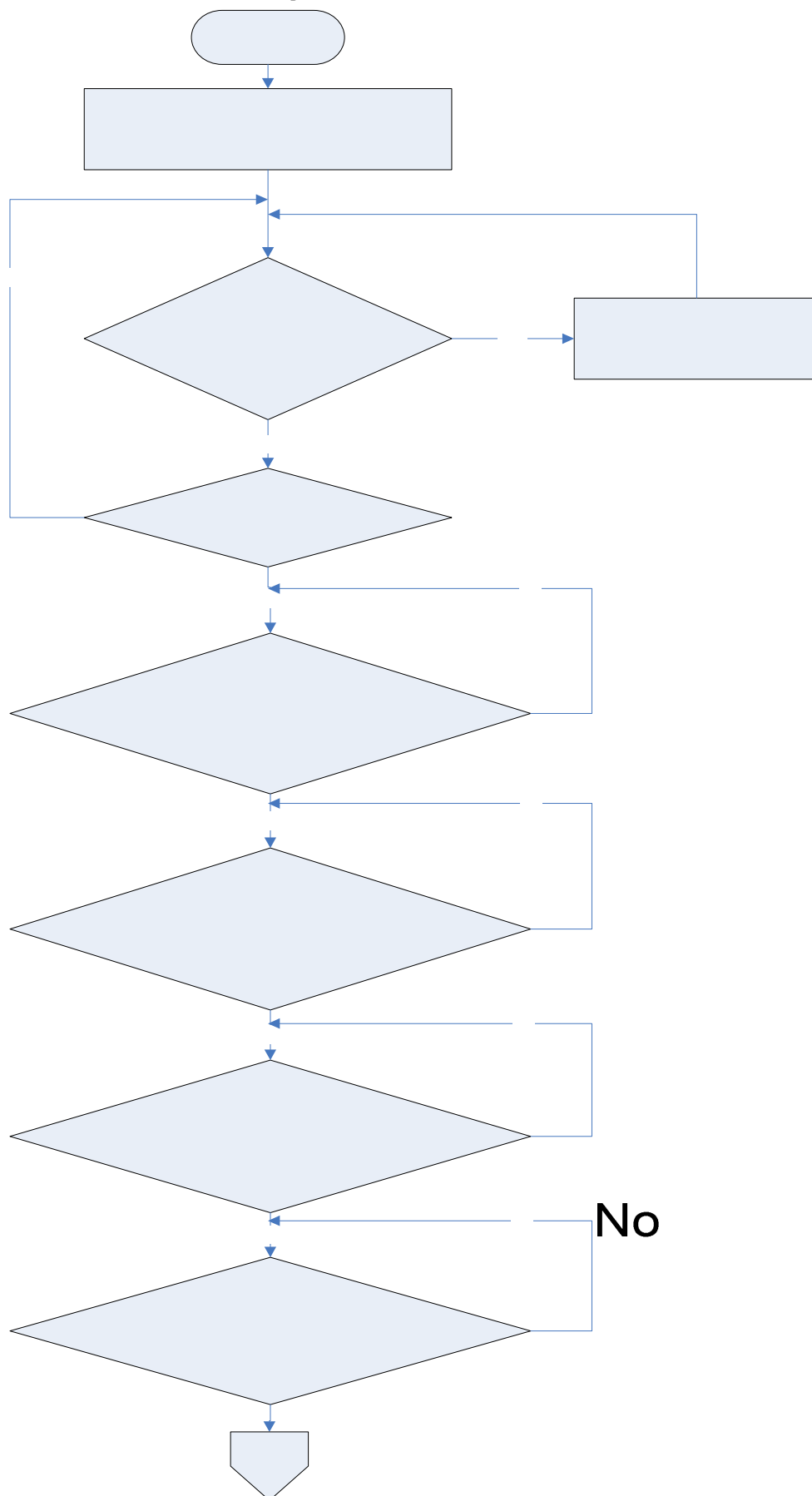


รูปที่ 3.15 Flow chart ของโปรแกรมการหาขนาดของดาว (ต่อ)

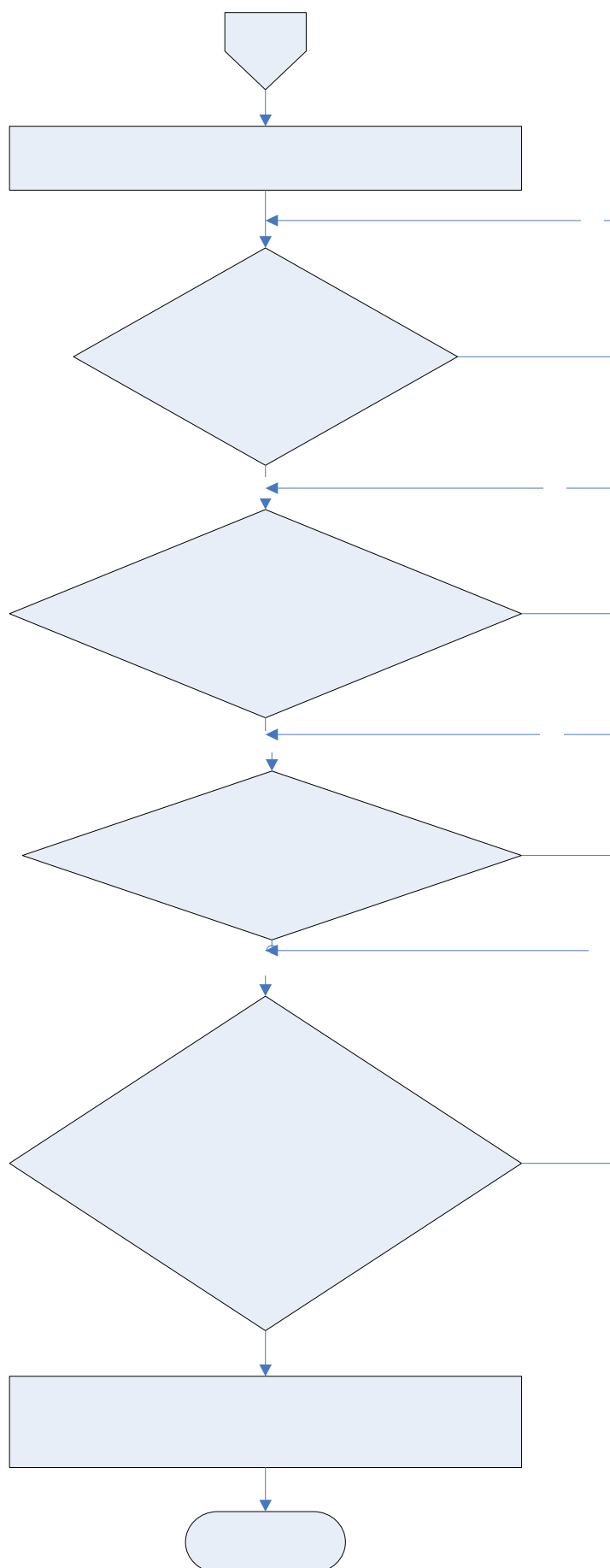


รูปที่ 3.16 Flow chart ของโปรแกรมการหาขนาดของดา (ต่อ)

โปรแกรมการหาขนาดของจุมูกและปาก



รูปที่ 3.17 Flow chart ของการหาขนาดจุมูกและปาก



รูปที่ 3.18 Flow chart ของการหาขนาดหมู่และปาก (ต่อ)